

Assignment

1. Follow the prescribed procedure to identify compiler errors in the given C program by carefully examining each statement, detecting all errors, classifying them as lexical, syntax, semantic errors, logical errors or runtime errors and rewriting the incorrect statements with their appropriate corrections.

```
#include <stdio.h>

int calculator()
{
    int 1a = 10, b =20;
    float d = 2.5;
    int x;
    x = 1a + d;
    int y = b - 20
    int z = b / y ;
    if ( 1a < b )
        small = b ;
    print("%d is smaller" b )
    return d;
}

int main()
{
    calculator();
    return 0;
}
```

2. Follow the prescribed procedure to identify the common prefixes in the following grammar, apply the left factoring technique step by step, and write the equivalent left-factored grammar.

```
S → aAB | aAC | aD | bA
A → cde | cdf | cdg | h
B → ij | ik
C → il | im
D → cde | cdh
```

3. Follow the prescribed procedure to construct the recursive descent parsing functions for the given grammar and demonstrate step-by-step parsing of the input string using those function. **Input string:** a a b b c d

```
S → A B C
A → a A | a
B → b B | b | ε
```

$C \rightarrow c C \mid d$

4. Follow the prescribed procedure to identify lexemes, corresponding patterns, and tokens from the given source code by scanning each statement and classifying every valid lexeme into its correct token category.

```
#include <stdio.h>
```

```
int main()  
{  
    int num = 25;  
    float value = 5.5;  
    if (num > value)  
        printf("Greater value");  
    return 0;  
}
```

5. Follow the prescribed procedure to determine whether the given grammar is LL(1) by eliminating left recursion (if any), performing left factoring (if required), computing FIRST and FOLLOW sets, and constructing the LL(1) parsing table.

$S \rightarrow S A B c \mid E f$

$A \rightarrow a A \mid \epsilon$

$B \rightarrow b B \mid b d$

$E \rightarrow a \mid \epsilon$

Assignment

1. Classify errors lexical, syntax, semantic, logical, runtime

```
# include <stdio.h>
```

```
int calculator()
```

```
{ int 1a = 10, b = 20;
```

```
float d = 2.5;
```

```
int x;
```

```
x = 1a + d;
```

```
int y = b - 20;
```

```
int z = b / y; [20/0; cannot divide by 0]
```

```
if (1a < b)
```

```
Small = b;
```

```
printf("%d is smaller than b);
```

```
return d;
```

```
}
```

```
int main()
```

```
{ calculator();
```

```
return 0;
```

```
}
```

Correct:

```
#include <stdio.h>
```

```
int calculator()
```

```
{ int a=10, b=20;
```

```
float d=2.5;
```

```
float x;
```

```
x=a+b;
```

```
int y=b-10;
```

```
int z=b/4;
```

```
int small;
```

```
if (a<b)
```

```
{ small=a;
```

```
printf ("%d is smaller", a);
```

```
return a;
```

```
}
```

```
int main()
```

```
{ calculator();
```

```
return 0;
```

```
}
```

2. Left factoring

$$S \rightarrow \underline{a}AB | \underline{a}Ac | \underline{a}D | bA$$

$$A \rightarrow \underline{c}de | \underline{c}df | \underline{c}dg | h$$

$$B \rightarrow ij | ik$$

$$c \rightarrow il | im$$

$$D \rightarrow cde | edh$$

Solve:

$$S \rightarrow aS' | bA$$

$$S' \rightarrow \underline{A}B | \underline{A}c | D$$

$$S' \rightarrow AS'' | D$$

$$S'' \rightarrow B | c$$

$$A \rightarrow \underline{c}dA' | h$$

$$A' \rightarrow e | f | g$$

$$B \rightarrow iB'$$

$$B' \rightarrow j | k$$

$$c \rightarrow ic'$$

$$c' \rightarrow l | m$$

$$D \rightarrow cD'$$

$$D' \rightarrow e | h$$



3. Recursive descent Parsing

$S \rightarrow ABC$

$A \rightarrow aA|a$

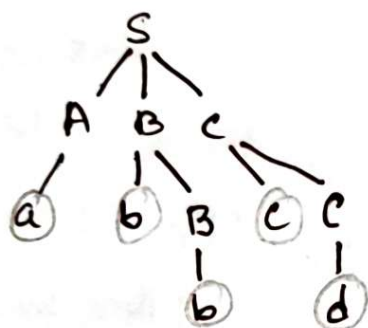
$B \rightarrow bB|b|e$

$C \rightarrow cC|d$

input string: abbed

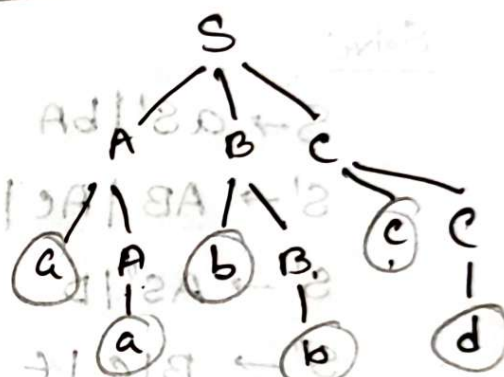
उत्तर इस प्रकार है :-

Solve:



= abbed

input string: aabbed



= aabbed

$A \rightarrow aA|a$

$B \rightarrow bB|b|e$

$C \rightarrow cC|d$

$A \rightarrow aA|a$

$B \rightarrow bB|b|e$

$C \rightarrow cC|d$

$A \rightarrow aA|a$

4. Identify lexemes, patterns & tokens

```
#include <stdio.h>
```

```
int main()
```

```
{ int num = 25;
```

```
float value = 5.5;
```

```
if (num > value)
```

```
printf ("Greater value");
```

```
return 0;
```

```
}
```

Token	Lexeme	Pattern
Identifier	num, value	letter followed by digits or letter B → B12
Keyword	int, float, if, return	reserved words
Operator	=, >	+, -, *, /, ==, %, ...
Number	25, 5.5	Integer value, float value Digit followed by 0 or more

5. LL(1) Parsing Table

	FIRST	FOLLOW
$S \rightarrow \dots Efs'$	$\{$	$\{$
$A \rightarrow aA \epsilon$	$\{a, \epsilon\}$	$\{$
$B \rightarrow bB bd$	$\{b, d\}$	$\{$
$E \rightarrow a \epsilon$	$\{a, \epsilon\}$	$\{$
$S' \rightarrow ABCS' \epsilon$	$\{$	$\{$

$S \rightarrow SABc|Ef$

$S \rightarrow Efs'$

$S' \rightarrow ABCS'| \epsilon$

left recursion

$A \rightarrow aA| \epsilon$

$B \rightarrow bB'$

$B' \rightarrow B|d$

left factoring

$E \rightarrow a| \epsilon$

	FIRST	FOLLOW
$S \rightarrow Efs'$	$\{a, f\}$	$\{\$ \}$
$s' \rightarrow ABes' \epsilon$	$\{a, b, \epsilon\}$	$\{b\}$
$A \rightarrow aA \epsilon$	$\{a, \epsilon\}$	$\{e\}$
$B \rightarrow bB'$	$\{b\}$	$\{c\}$
$B' \rightarrow B d$	$\{b, d\}$	$\{f\}$
$E \rightarrow a \epsilon$	$\{a, \epsilon\}$	

Parse Table:

	\$	a	b	c	d	f
S		$S \rightarrow Efs'$				$S \rightarrow Efs'$
s'	$s' \rightarrow \epsilon$	$s' \rightarrow ABes'$	$s' \rightarrow ABes'$			
A		$A \rightarrow aA$	$A \rightarrow \epsilon$			
B			$B \rightarrow bB'$			
B'			$B' \rightarrow B$		$B' \rightarrow d$	
E		$E \rightarrow a$				$E \rightarrow \epsilon$